

Automation Frameworks and Best Practices for Cloud-Native Database Lifecycle Management

Jagadeesh Kola

College of Arts and Sciences, Sacred Heart University, Fairfield, Connecticut, USA

ARTICLE INFO

Article History:

Accepted : 25 June 2024

Published : 13 July 2024

Publication Issue :

Volume 11, Issue 4

July-August-2024

Page Number :

676-686

ABSTRACT

The migration to cloud-native frameworks makes it imperative to effectively manage database lifecycle across organizations. Also, well developed and automated applications and web systems are needed for effectiveness and scalability of database that meets global standard. This study attempts a comprehensive review of best practices and automation frameworks for effective management of cloud-native database lifecycle. These include scaling, decommissioning, configuration, monitoring and provisioning. The study analyzes orchestration of container and the place of infrastructure as Code (IaC) in the automation of database operations in both public and private spheres of hybrid cloud-native environment. It also analyzes prominent managed services like Google Cloud SQL and Amazon RDS and tools like Kubernetes Operators and Terraform. In the course of the analysis, their strengths and weaknesses are highlighted. The article presents some practicable best practices capable of optimizing performance, ensuring regulatory compliance and safety, and minimizing operational bottlenecks. In all, the study has uniquely offered a realizable guidance to the concerned professionals on how to effectively streamline and manage database operations in changing cloud-native ecological spaces.

Keywords : Cloud-native, Database lifecycle, Automation frameworks, Best practices, Management

Introduction

The migration to cloud-native frameworks makes it imperative to effectively manage database lifecycle across organizations. Also, well developed and automated applications and web systems are needed for effectiveness and scalability of database that meets global standard (Ogirri et al., 2024; Mwiinga,

2023; Oguntola, 2023). The management of cloud infrastructure has not been easy (Chippagiri & Ravula, 2021). The migration has been informed by technological advancements in cloud computing (Akinloye & Shawana, 2023; Hussien et al., 2023; Ionescu et al., 2023; L'Esteve, 2023; Malallah et al., 2023; Shawana, 2022). It has led to a sporadic shift from the then container-based environment and

operation of applications as microservices that were controlled based on elastic infrastructure to orchestration (Kumar et al., 2019; Kumar, 2014; Kumar et al., 2010). Microservices frameworks for implementation include Molecular (JavaScript with Node.js), Go Micro (Go), and Spring Boot/Spring Cloud (Java) (Theodoropoulos et al., 2023; Kumar et al., 2013).

Given that microservices have various components, failure and detection inaccuracies can arise from the intricate relationships between/among services and the rapid modifications of microservices-based system versions (Ahmad et al., 2021; Dittakavi, 2022; Qiu et al. 2020). Thus, to avert failures and inadequacies, automation frameworks and best practices have to be leveraged for effective management of cloud-native database lifecycle. There is no doubt that orchestration is an impactful development in cloud computing (Kumar et al., 2010). It is an essential element of cloud-native applications, which allows for effective resource management, scalability, scheduling, and anomaly detection, among other functions (Chowdhury et al. 2020). The advent of digital businesses brought to place the migration of enterprise apps to cloud platforms.

Therefore, this study is aimed at making an ample review of best practices and automation frameworks for effective management of cloud-native database lifecycle. The frameworks and best practices include scaling, decommissioning, configuration, monitoring and provisioning. The study is concerned with analyzing the orchestration of container and the place or role of infrastructure as Code (IaC) in the automation of database operations in both public and private spheres. Its focused cloud-native environment is that of hybrid. It is also concerned with exploring famous managed services like Google Cloud SQL and Amazon RDS, and tools like Kubernetes Operators and Terraform (Dhanagari, 2023; Boddu, 2022; Eshwari et al., 2020; Kumar et al., 2010).

Automation Frameworks

With the innovative advancements in cloud computing, microservice architecture (MSA) has come into existence as a prominent web application design (Dinh-Tuan et al., 2020). MSA is used to build and implement complex software systems. The use of MSA breaks them into single-function service components. The MSA is comparable with the erstwhile service-oriented architecture (SOA). Cloud-based apps have become popular because of accessibility and effectiveness of computing resources and cost-effectiveness of hosting. The effectiveness and scalability of any software systems, including effective development and management of the lifecycle of cloud-native database, require performance evaluation, monitoring and control, and scalability testing.

These requirements are needed to ensure that the SLA standards for cloud services are met, and reliable groundwork for futuristic optimization is provided (Atmaca et al. 2016). The following diagram shows cloud computing architecture that plays a critical role in automation frameworks and best practices for cloud-native database lifecycle management:

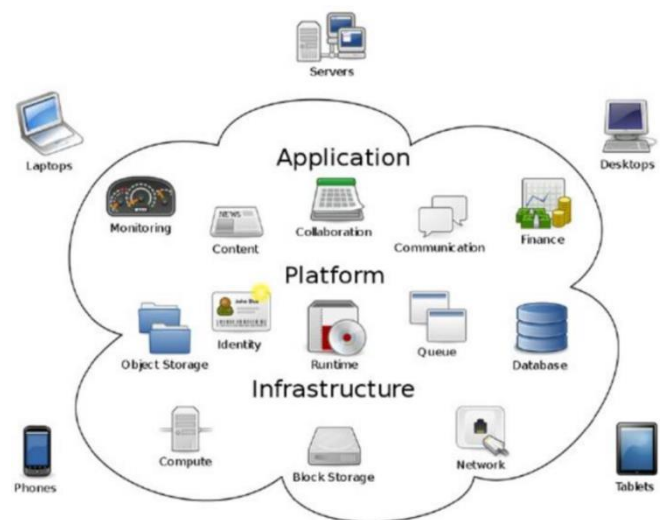


Fig. 1 : Cloud Computing Architecture

Source: Ang'udi (2023)

As Cappaert and Redei (2020) affirm, to address the aforementioned challenges, vigorous frameworks and best practices ought to be deployed for building

scalable and resilient cloud-native systems. It is to that end this study engages with automation frameworks and best practices for cloud-native database lifecycle management. There are different automation frameworks. These include configuration management techniques, Infrastructure as Code (IaC) tools, database migration techniques, tools for monitoring and observability, and continuous integration/continuous deployment (CI/CD) techniques (Peram, 2024; Kosińska et al., 2023; Hasan et al., 2015; Hasan et al., 2012). Organizations need CI and Delivery Systems (DS) in order to implement Continuous Deployment. In other words, without CI and DS, CD cannot be implemented (Hakli et al., 2018). The relationship among these three is illustrated in the following diagram:

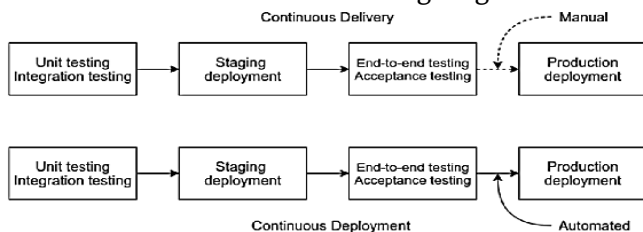


Fig. 2 : Continuous Integration, Delivery and Deployment in Intersection

Source: Hakli et al. (2018)

Each of these five sets of tools has different tools. Accordingly, the configuration management tools have Chef/Puppet and Ansible. Ansible is an open-source automation tool used for configuring and managing databases in order to attain environmental consistency (Ganesan, 2020). That is, consistency across environments can be set up and ensured sustainably by using Ansible. Effective management and configuration of database servers can be automated by Chef/Puppet. The automation ensures regulatory compliance among organizations that apply the tools. That is, the automation ensures compliance with legislations. The following diagram represents cloud computing environment needed for effective management and configuration of database servers, automation frameworks and best practices for cloud-native database lifecycle management:

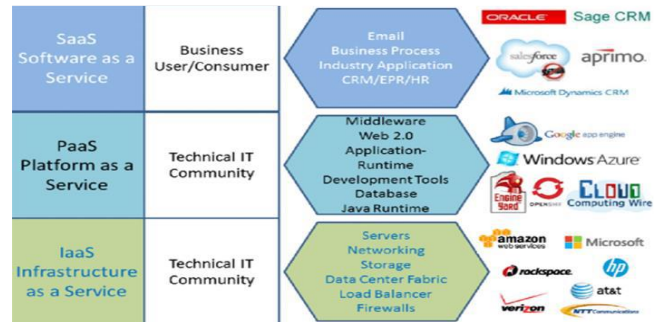


Fig. 3: Cloud-Native Environment

Source: Ang'udi (2023)

Infrastructure as Code (IaC) tools makes it possible for cloud infrastructure to be defined and provided. In defining and providing them, a declarative configuration language is used. IaC can help manage databases and cloud resources at a significant extent, thereby guaranteeing optimization and appropriate design. It effectively manages cloud resources and automates their lifecycle. IaC guarantees scalability and flexibility of applications (Wurster et al., 2020). It lessens the chances of configuration drift by making sure that infrastructure is reliably distributed across environments (Kumar et al., 2015). Also, with IaC, databases can be modeled and Amazon Web Service resources set up using templates (Brunner et al., 2015).

The process of modeling and setting up databases and web service resources is regarded as AWS CloudFormation (Dhanagari, 2023; Atmaca et al., 2016). This is an integral part of the Infrastructure as Code (IaC) tools. Azure Resource Manager (ARM) is another tool of the Infrastructure as Code (IaC) tools. It is similar to CloudFormation. It enables one to apply and manage Azure resources using templates (Dhanagari, 2023). GitLab CI/CD and Jenkins are major tools of the continuous integration and continuous deployment (CI/CD) tools (Ganesan, 2020). GitLab CI/CD can be combined with version control for the automation of databases. This automation focuses migration and deployment of databases. By implication, the GitLab CI and CD function in collaboration with version control.

Similarly, the function of Jenkins supplements that of GitLab CI/CD. This is because it is the tool

responsible for the automation of the deployment of database changes. It serves as the pipeline of a CI/CD. CI/CD pipelines promote cooperation and fast iteration. These cloud-native applications and others usually have the ability to automatically adapt to evolving workloads. Consequently, they ensure best possible performance and cost-effectiveness (Kumar et al., 2015). The following diagram highlights use elements of cloud-native management processes:

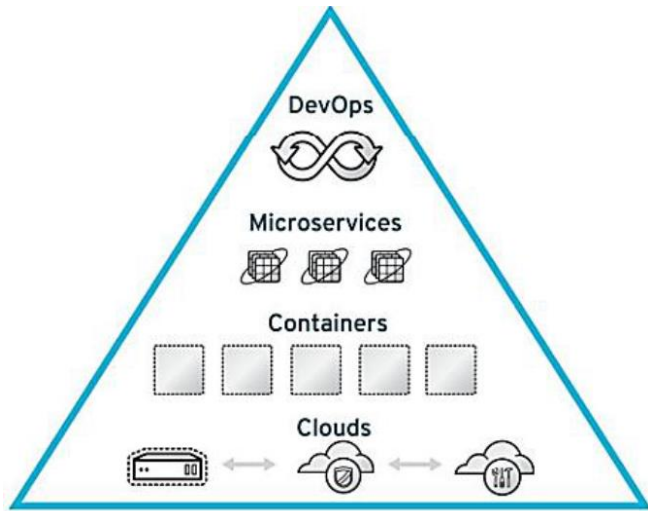


Fig. 4 : A pyramid of modern cloud-native applications

Source: Chippagiri and Ravula (2021)

Chippagiri and Ravula (2021) point out resilience, scalability, and elasticity as the fundamental principles of cloud-native development. By implication, the principles apply to the management of cloud-native database lifecycle using automation frameworks and best practices. Their study emphasizes that continuous integration and deployment (CI/CD), security-focused DevSecOps and infrastructure as code (IaC) practices are crucial for effective cloud computing and cloud-native database lifecycle management (Kumar et al., 2009). More so, database migration tools include flyway and liquibase, which are similar. Flyway is a version control tool for databases, which allows for the management of schema migrations and automates the deployment of changes. In the same vein, liquibase is a tool automates database migrations and manages database changes.

Apart from the foregoing explained automation frameworks, there are monitoring and observability tools. These comprise Datadog/New Relic and Prometheus/Grafana. Chippagiri and Ravula (2021) demonstrate that AWS CloudWatch, Prometheus and Kubernetes are viable frameworks for optimization, orchestration and monitoring of cloud-native systems. They emphasize that safe and high-performance applications can be built by any organizations in the industry that adopt cloud-native technologies. The Datadog/New Relic tools proffer clues to understating performance and metrics in managing cloud-native database lifecycle to attain best practices and optimized automation.

In other words, they provide valuable insights into database performance and its metrics. Scaling decisions are made based on the metrics. The decisions are automated by Datadog/New Relic. Database health and performance are monitored effectively by Prometheus/Grafana (Chippagiri & Ravula, 2021). These functions of the Prometheus/Grafana pave way for the automation of dashboards and alerts. It follows that additional to optimizing performance, Prometheus/Grafana maintains the health of databases, allowing for efficiency, scalability, protection and realization of best practices in the management of cloud-native database lifecycle.

Best Practices for Cloud-Native Database Lifecycle Management

Best practices include automated backups, version control, automated testing, environment consistency, scaling automation, monitoring and alerts, security automation, documentation and change management, cost management, and disaster recovery planning (Peram, 2024; Ugwueze, 2024). Industrial cloud standards, cloud-native application design approaches, and public CSP are examined by Kratzke and Peinl (2016). Their analysis demonstrates that majority of cloud service categories appear to encourage vendor lock-in scenarios that may be particularly troublesome for

structures of certain businesses. Similarly, automated tool for OpenStack Kubernetes cluster deployment and monitoring are provided by Astyrakakis et al. (2019). They developed Kubernetes clusters were deployed with very short overall timeframes, whereby Kubernetes HPA enabled took around 11 minutes to validate, while a containerized application having the HPA took about 3 minutes to get deactivated.

Monitoring of activities is essential in order to detect abnormal or unauthorized activities that could cause data breaches, whereas encryption makes sure that data remains indecipherable even if accessed. As such, the implementation of comprehensive Data Loss Prevention (DLP) solutions, organizations can strengthen their security department and encourage confidence in their level of data protection and

confidentiality (Selvan et al., 2021). DLP is an essential strategy for protecting critical information from unintentional and intentional exposure. It entails deploying tools like activity monitoring and encryption for monitoring and regulation of the flow of oversensitive data in the cloud environment (Nyangaresi, 2022; Alsuwaie et al., 2021; Turobova et al., 2021; Stallings, 2020).

On their part, Imadali and Bousselmi (2018) provide a software platform that enables cloud computing, MNOs to make their networking resources, and mobile services available to 5GaaS over-the-top companies. Some other literatures are tabulated hereafter for further evidence for cloud-native frameworks for resilient and scalable web applications:

Table 1 : Comparing cloud-native development and frameworks for scalable and resilient web applications

Reference	Study On	Methodology	Key Findings	Challenges	Limitations
Brunner et al., 2015	Case study of a business application running as a Cloud-Native Application (CNA)	Experimental evaluation of CNA advantages using Dynamite auto-scaler on Vagrant, OpenStack, and Amazon EC2 testbeds.	Demonstrated CNA requires minimal additional engineering, and Dynamite enhances auto-scaling for containerized applications.	Resource management in varied cloud environments.	Limited scope to specific experimental testbeds (e.g., Vagrant, OpenStack, Amazon EC2)
Chang and Fink, 2017	Visualisation of cloud execution logs for program understanding and documentation generation.	Developed a tool with timeline visualisation, JSON summarisation, and interaction techniques for runtime data analysis.	Improved understanding of serverless cloud applications' composition, performance, and data flow with positive feedback from expert developers.	Ensuring usability across diverse cloud applications.	Initial feedback limited to expert developers; broader validation not covered.

Kratzke and Peinl, 2016	Design principles, public cloud services, and industrial cloud standards for cloud-native apps.	Developed a reference model relying on standardised IaaS services to address vendor lock-in concerns.	Highlighted vendor lock-in risks in public cloud services and proposed a model for codifying cloud technologies and guiding enterprise architectures.	Overcoming vendor lock-in in real-world implementations.	Focuses on IaaS and lacks detailed discussion of PaaS and SaaS contexts.
Astyrakakis et al., 2019	Automated deployment and monitoring of Kubernetes clusters over OpenStack.	Developed and evaluated an automated tool for Kubernetes deployment and validation with HPA-enabled/disabled setups.	Achieved faster Kubernetes cluster deployment times and efficient validation processes with notable time reductions compared to manual approaches.	Adapting the tool for larger, more complex deployments.	Limited evaluation scope to Kubernetes clusters and specific configurations.
Imadali and Bousselmi, 2018	Cloud-native software platform enabling MNOs to expose assets for 5GaaS.	Developed an open-source Cloud Native VNF API design and evaluated feasibility from a standards perspective.	Provided a viable prototype for 5GaaS with a design enabling integration of networking resources, mobile services, and cloud computing.	Balancing standardisation and innovation requirements.	Focused on 5GaaS-specific use cases; broader applicability to other industries not demonstrated.

Source: Chippagiri and Ravula (2021)

Accordingly, version control comprises migration scripts and store database schema. Git is a typical example of version control systems. Migration scripts and store database schema help track changes and ease collaboration. Automated backup strategies

guarantee regular backing up of data, allowing for effective security and management of data (Dhanagari, 2023). Leveraging automated backup strategies for cloud-native database lifecycle management also prevents failures, allows for effective storage of data, and optimizes automation

and implementation of best practices for cloud-native database lifecycle management. Environment consistency involves using automation techniques like IaC to make sure consistent development, testing, and production environments, where risks are reduced to the barest minimum (Cappaert & Redei, 2020).

With automation framework and best practice for cloud-native database lifecycle management, environment-specific issues are easily identified and mitigated, thereby allowing for environmental consistency. Monitoring and alerts are means of monitoring and setting up alerts for database performance metrics (Atmaca et al., 2016). With these, cloud-native database lifecycle management systems are appropriately automated and monitored; database performance metrics are measured, determined and monitored; and error rates are detected, predicted and prevented (Ganesan, 2020; Qiu et al., 2020). Also, monitoring and alerts automation frameworks and best practices for cloud-native database lifecycle management that question performance rate and weigh the extent of resource utilization (Kumar & Chan, 2011). With automated testing, database changes are made and realized. These include the unit tests for stored processes and integration tests. These drive interactions in the application-database of cloud-native database lifecycle.

Scaling automation involves using auto-scaling features made available by cloud providers to automatically adjust resources of database on the basis of load (Linthicum, 2017). These automation frameworks work on database based on load to allow for scalability and the adjustment of database resources (Peram, 2024; Tiwari & Bhatt, 2022). The resources are provided automatically by cloud providers. This means that for these frameworks to carry out these functions, the cloud providers must have acted by providing the database resources for adjustment. The cloud providers cannot adjust the load of the database, but scaling automation could

do so by using auto-scaling features provided by the cloud providers (Brunner et al., 2015). This process highlights the systemic functionality of the various components of cloud-native database lifecycle.

In managing cloud-native database lifecycle, different components must act collaboratively and procedurally, with one taking up the actions stopped by the other that has reached its limit of actions until the process is complete. Automation security is another framework for attaining best practices for cloud-native database lifecycle management (Gagged & Murugaiyan, 2023; Chippagiri & Ravula, 2021). The best practices automated by security automation include encrypting data in transit and at rest, accessing control management systems, and regular scanning of vulnerability (Saranya et al., 2023; Sunday & Olufunminiyi, 2023; Umran et al., 2023; Jana & Oprea, 2019). As automation frameworks and best practices for cloud-native database lifecycle management, proper documentation and appropriate management of change involve maintaining clear documentation of database changes and automating the generation of change logs in order to aid audits and regulatory compliance (Ang'udi, 2023).

Also, disaster recovery planning is a framework and best practice for cloud-native database lifecycle management, which automates the processes of disaster recovery (Alsamhi et al., 2022). The processes include recovery testing and failover mechanisms, which are used to ensure the continuity of businesses. The implication of the foregoing is that this framework for attaining best practices for cloud-native database lifecycle management guarantees the sustenance of businesses and the optimization of organizations' security systems. This framework helps mitigate disaster and fosters recovery optimization. Finally, cost management is a viable framework for attaining best practices for cloud-native database lifecycle management (Dhanagari, 2023; Ghandour et al., 2023; Baldini et al., 2017; Toffetti et al., 2017). It

entails managing costs by implementing automated cost monitoring and optimization strategies. The use of these strategies allows for effective management of the expenses on cloud database.

Conclusion

The importance of adherence to and ensuring best practices in various settings cannot be overemphasized. In cloud-native database lifecycle management, best practices are also imperative. It is to that end this study has explored and shown automation frameworks and best practices for cloud-native database lifecycle management. In the course of its analysis, the study highlights the strengths and weaknesses of the orchestration of container and the place of infrastructure as Code (IaC) in the automation of database operations in both public and private spheres of hybrid cloud-native environment. The study analyzes orchestration of container and the place of infrastructure as Code (IaC) in the automation of database operations in both public and private spheres of hybrid cloud-native environment. It also analyzes prominent managed services like Google Cloud SQL and Amazon RDS and tools like Kubernetes Operators and Terraform.

Finally, the article presents some practicable best practices that are capable of optimizing performance, ensuring regulatory compliance and safety, and minimizing operational bottlenecks. There is no doubt that the study has uniquely offered a realizable guidance to the concerned professionals on how to effectively streamline and manage database operations in changing cloud-native ecological spaces. By so doing, it has made a novel contribution to discourses on cloud-native database lifecycle management and the imperative of automation frameworks and best practices for cloud-native database lifecycle management. It offers stakeholders in the industry valuable guides to effective management of cloud-native database lifecycle and how to uphold and ensure effective

automation and adherence to best practices in changing cloud-native ecological spaces in various settings.

REFERENCES

- [1]. Ahmad, W., Rasool, A., Javed, A. R., Baker, T., & Jalil, Z. (2021). Cyber security in IoT-based cloud computing: A comprehensive survey. *Electronics*, 11(1), 16.
- [2]. Akinloye, A., & Shawana, T. A. (2023). Gas caps detection, hydrocarbon recovery and environmental sustainability. *Journal of Applied Ecology and Environmental Design (JAEED)*, 2(2), 118-130.
- [3]. Alsamhi, S. H., Shvetsov, A.V., Kumar, S., Shvetsova, S. V., Alhartomi, M. A., Hawbani, A., Rajput, N. S., Srivastava, S., Saif, A., & Nyangaresi, V. O. (2022). UAV computing-assisted search and rescue mission framework for disaster and harsh environment. *Drones*, 6(7), 154.
- [4]. Alsawaie, M. A., Habibnia, B., & Gladyshev, P. (2021). Data leakage prevention adoption model & DLP maturity level assessment. In 2021 International Symposium on Computer Science and Intelligent Controls (ISCSIC), 396-405.
- [5]. Ang'udi, J. J. (2023). Security challenges in cloud computing: A comprehensive analysis. *World Journal of Advanced Engineering Technology and Sciences*, 10(02), 155-181. DOI: <https://doi.org/10.30574/wjaets.2023.10.2.0304>
- [6]. Astyrakakis, N., Nikoloudakis, Y., Kefaloukos, I., Skianis, C., Pallis, E., & Markakis, E. K. (2019). Cloud-Native application validation & stress testing through a framework for auto-cluster deployment. In 2019 IEEE 24th International Workshop on Computer Aided Modeling and Design of Communication Links and Networks (CAMAD), 1-5. doi: 10.1109/CAMAD.2019.8858164.
- [7]. Atmaca, T., Begin, T., Brandwajn, A., & Castel-Taleb, H. (2016). Performance evaluation of cloud computing centers with general arrivals

- and service. IEEE Trans. Parallel Distrib. Syst., 27(8), 2341–2348. doi: 10.1109/TPDS.2015.2499749.
- [8]. Baldini, I. et al., (2017). Serverless computing: Current trends and open problems. In Research Advances in Cloud Computing, doi: 10.1007/978-981-10-5026-8_1.
- [9]. Boddu, B. (2022). Cloud DBA strategies for SQL and NOSQL data management for business-critical applications. International Journal of Core Engineering & Management, 7(1), 113-127.
- [10]. Brunner, S., Blochlinger, M., Toffetti, G., Spillner, J., & Bohnert, T. M. (2015). Experimental evaluation of the cloud-native application design. Proc. - 2015 IEEE/ACM 8th Int. Conf. Util. Cloud Comput. UCC, 488–493. doi: 10.1109/UCC.2015.87.
- [11]. Cappaert, P., & Redei, A. (2020). A scalable cloud native platform for interactive museum exhibits. In EPIc Series in Computing, doi: 10.29007/6b3j.
- [12]. Chang, K. S. P., & Fink, S. J. (2017). Visualizing serverless cloud application logs for program understanding. In Proceedings of IEEE Symposium on Visual Languages and Human-Centric Computing, VL/HCC, doi: 10.1109/VLHCC.2017.8103476.
- [13]. Chippagiri, S., & Ravula, P. (2021). Cloud-native development: Review of best practices and frameworks for scalable and resilient web applications. International Journal of New Media Studies (IJNMS), 8(2), 13-21.
- [14]. Chowdhury, R., Talhi, C., Ould-Slimane, H., & Mourad, A. (2020). A framework for automated monitoring and orchestration of cloud-native applications. In 2020 International Symposium on Networks, Computers and Communications (ISNCC), IEEE, 1–6. doi: 10.1109/ISNCC49221.2020.9297238.
- [15]. Dhanagari, M. R. (2023). MongoDB in the cloud: Leveraging cloud-native features for modern applications. International Journal of Science and Research Archive, 10(02), 1297-1313. DOI: <https://doi.org/10.30574/ijrsra.2023.10.2.1089>
- [16]. Dinh-Tuan, H., Mora-Martinez, M., Beierle, F., & Garzon, S. R. (2020). Development frameworks for microservice-based applications: Evaluation and comparison. In ACM International Conference Proceeding Series, doi: 10.1145/3393822.3432339.
- [17]. Dittakavi, R. S. Evaluating the Efficiency and Limitations of Configuration Strategies in Hybrid Cloud Environments. International Journal of Intelligent Automation and Computing, 5(2), 29-45.
- [18]. Eshwari, H. M., Rekha, B. S., & Srinivasan, G. N. (2020). Hybrid cloud technologies: Dockers, containers and kubernetes. Int. Res. J. Eng. Technol., 7(6), 7628–7634.
- [19]. Ganesan, P. (2020). DevOps automation for cloud native distributed applications. Journal of Scientific and Engineering Research, 7(2), 342-347.
- [20]. Ghandour, O., El Kafhali, S., & Hanini, M. (2023). Computing resources scalability performance analysis in cloud computing data center. Journal of Grid Computing, 21(4), 61.
- [21]. Hakli, A., Taibi, D., & Systa, K. (2018). Towards cloud native continuous delivery: An industrial experience report. 2018 Proc. - 11th IEEE/ACM Int. Conf. Util. Cloud Comput. Companion, UCC Companion, 314–320 doi: 10.1109/UCC-Companion.2018.00074.
- [22]. Hasan, M. Z., Fink, R. M., Suyambu, R., & Baskaran, M. K. (2012). Assessment and improvement of intelligent controllers for elevator energy efficiency. In IEEE International Conference on Electro Information Technology, doi: 10.1109/EIT.2012.6220727.
- [23]. Hasan, M. Z., Fink, R., Suyambu, M. R., Baskaran, M. K., James, D., & Gamboa, J. (2015). Performance evaluation of energy efficient intelligent elevator controllers. In IEEE International Conference on Electro Information Technology, doi: 10.1109/EIT.2015.7293320.
- [24]. Hussien, Z. A., Abdulmalik, H. A., Hussain, M. A., Nyangaresi, V. O., Ma, J., Abduljabbar, Z. A., & Abduljaleel, I. Q. (2023). lightweight integrity

preserving scheme for secure data exchange in cloud-based Iot systems. *Applied Sciences*, 13(2), 691.

- [25]. Imadali, S., & Bousselmi, A. (2018). Cloud native 5g virtual network functions: Design principles and use cases. In *Proceedings - 8th IEEE International Symposium on Cloud and Services Computing, SC2*. doi: 10.1109/SC2.2018.00019.
- [26]. Ionescu, S. A., & Diaconita, V. (2023). Transforming Financial decision-making: The interplay of AI, cloud computing and advanced data management technologies. *International Journal of Computers Communications & Control*, 18(6).
- [27]. Jana, I., & Oprea, A. (2019). AppMine: behavioral analytics for web application vulnerability detection. *arxiv*, doi: 10.48550/arXiv.1908.01928.
- [28]. Kosińska, J., Baliś, B., Konieczny, M., Malawski, M., & Zieliński, S. (2023). Toward the observability of cloud-native applications: the overview of the state-of-the-art. *IEEE Access*, 11, 73036-73052. Doi: 10.1109/ACCESS.2023.3281860
- [29]. Kratzke, N., & Peinl, R. (2016). ClouNS-a cloud-native application reference model for enterprise architects. In *Proceedings – 2016, IEEE International Enterprise Distributed Object Computing Workshop, EDOCW*, doi: 10.1109/EDOCW.2016.7584353.
- [30]. Kumar, V. V. (2014). An interactive product development model in remanufacturing environment : Achaos-based artificial bee colony approach. *Missouri University of Science and Technology*, [Online]. https://scholarsmine.mst.edu/cgi/viewcontent.cgi?article=8243&context=masters_theses
- [31]. Kumar, V. V., Yadav, S. R., Liou, F. W., & Balakrishnan, S. N. (2013). A digital interface for the part designers and the fixture designers for a reconfigurable assembly system. *Math. Probl. Eng.*, doi: 10.1155/2013/943702.
- [32]. Kumar, V., & Chan, F. T. S. (2011). A superiority search and optimisation algorithm to solve RFID and an environmental factor embedded closed loop logistics model. *Int. J. Prod. Res.*, 49(16). doi: 10.1080/00207543.2010.503201.
- [33]. Kumar, V. V., Pandey, M. K., Tiwari, M. K., & Ben-Arieh, D. (2010). Simultaneous optimization of parts and operations sequences in SSMS: A chaos embedded Taguchi particle swarm optimization approach. *J. Intell. Manuf.*, doi: 10.1007/s10845-008-0175-4.
- [34]. Kumar, V. V., Sahoo, A., & Liou, F. W. (2019). Cyber-enabled product lifecycle management: A multi-agent framework. In *Procedia Manufacturing*, doi: 10.1016/j.promfg.2020.01.247.
- [35]. Kumar, V. V., Tripathi, M., Pandey, M. K., & Tiwari, M. K. (2009). Physical programming and conjoint analysis-based redundancy allocation in multistate systems: A Taguchi embedded algorithm selection and control (TAS&C) approach. *Proc. Inst. Mech. Eng. Part O J. Risk Reliab.*, 223(3), 215–232. doi: 10.1243/1748006XJRR210.
- [36]. L'Esteve, R. C. (2023). New horizons in distributed cloud computing. In *The cloud leader's handbook: Strategically innovate, transform, and scale organizations* (123-134). Apress.
- [37]. Malallah, H. S., Qashi, R., Abdulrahman, L. M., Omer, M. A., & Yazdeen, A. A. (2023). Performance analysis of enterprise cloud computing: A review. *Journal of Applied Science and Technology Trends*, 4(01), 01-12.
- [38]. Mwiinga, P. (2023). Privacy-preserving technologies: Balancing security and user privacy in the digital age. *Int. J. Sci. Res. Publ*, [Online]. <https://zenodo.org/records/10406538>
- [39]. Nyangaresi, V. O. (2022). Lightweight anonymous authentication protocol for resource-constrained smart home devices based on elliptic curve cryptography. *Journal of Systems Architecture*, 133, 102763.
- [40]. Ogirri, K. O., Akinloye, A., & Oguntona, T. I. (2024). Machine learning techniques, gas emission mitigation, and environmental safety.

- Journal of Science Innovation & Technology Research (JSITR), 5(9), 255-267.
- [41]. Oguntona, T. I. (2023, March). Mitigating the impact of high energy prices on UK households using machine learning techniques. *International Journal of Modelling & Applied Science Research*, 27(9), 215-230.
- [42]. Peram, P. (2024). Optimizing cloud computing performance: a comprehensive framework of strategies and best practices. *International Journal of Engineering and Technology Research (IJETR)*, 9(2), 387-396. DOI: <https://doi.org/10.5281/zenodo.13851330>
- [43]. Peram, P. (2024). Optimizing cloud computing performance: a comprehensive framework of strategies and best practices. *International Journal of Engineering and Technology Research (IJETR)*, 9(2), 387-396. DOI: <https://doi.org/10.5281/zenodo.13851330>
- [44]. Qiu, J., Du, Q., Yin, K., Zhang, S., & Qian, C. (2020). Applied sciences: A causality mining and knowledge graph based method of root cause diagnosis for performance. *Anomaly in Cloud Applications*, doi: 10.3390/app10062166.
- [45]. Saranya, N., Sakthivadivel, M., Karthikeyan, G., Rajkumar, R.. (2023). Securing the cloud: An empirical study on best practices for ensuring data privacy and protection. *International Journal of Engineering and Management Research*, 13(2), 46-9.
- [46]. Selvan, M. P., Sowmith, R. S., Dheeraj, P., & Jancy, S.(2021). High secured data access and leakage detection using attribute-based encryption. In *Advances in Electronics, Communication and Computing: Select Proceedings of ETAEERE 2020 2021* (433-445).
- [47]. Stallings, W. (2020). Data loss prevention as a privacy-enhancing technology. *Journal of Data Protection & Privacy*, 3(3), 323-33.
- [48]. Sunday, A. E., & Olufunmiyi, O. E. (2023). An efficient data protection for cloud storage through encryption. *International Journal of Advanced Networking and Applications*, 14(5), 609-18.
- [49]. Theodoropoulos, T., Rosa, L., Benzaid, C., Gray, P., Marin, E., Makris, A., Cordeiro, L., Diego, F., Sorokin, P., Girolamo, M.D., et al. (2023). Security in Cloud-Native Services: A Survey. *J. Cybersecur. Priv.*, 3, 758-793. <https://doi.org/10.3390/jcp3040034>
- [50]. Theodoropoulos, T., Rosa, L., Benzaid, C., Gray, P., Marin, E., Makris, A., Cordeiro, L., Diego, F., Sorokin, P., Girolamo, M. D. et al. (2023). Security in cloud-native services: a survey. *J. Cybersecur. Priv.*, 3, 758-793. <https://doi.org/10.3390/jcp3040034>
- [51]. Tiwari, S., & Bhatt, C. (2022). A comprehensive study on cloud computing: Architecture, load balancing, task scheduling and meta-heuristic optimization. In *International Conference on Intelligent Cyber Physical Systems and Internet of Things*, (137-162). International Publishing.
- [52]. Toffetti, G., Brunner, S., Blöchliger, M., Spillner, J., & Bohnert, T. M. (2017). Self-managing cloud-native applications: Design, implementation, and experience. *Futur. Gener. Comput. Syst.*, doi: 10.1016/j.future.2016.09.002.
- [53]. Turobova, G. O., Djangazova, Q. A., & Ganikhodjayeva, D. Z. (2021). Data loss prevention and challenges faced in their deployments. *Oriental renaissance: Innovative, Educational, Natural and Social Sciences*, 1(9), 176-82.
- [54]. Ugwueze, V. U. (2024). Cloud native application development: Best practices and challenges. *International Journal of Research Publication and Reviews*, 5(12), 2399-2412. DOI: <https://doi.org/10.55248/gengpi.5.1224.3533>
- [55]. Umrans, S. M., Lu, S., Abduljabbar, Z. A., & Nyangaresi, V. O. (2023). Multi-chain blockchain based secure data-sharing framework for industrial IoTs smart devices in petroleum industry. *Internet of Things*. 24, 100969.
- [56]. Wurster et al., M. (2020). The essential deployment metamodel: A systematic review of deployment automation technologies. In *Software-Intensive Cyber-Physical Systems*, doi: 10.1007/s00450-019-00412-x.